Lab10

08/04/2026

Hashing

A hash table stores data in an array using a hash function to find the slot for each key. When two keys map to the same slot, a collision occurs. **Open Addressing** handles this by looking for the next available slot within the same array. **Quadratic Probing** is one way to find that next slot. Instead of checking the very next slot (which causes long chains of occupied slots in linear probing), the gap between probes grows with each attempt. The probe function is $h(k,\, i) = (h'(k) + C_1 \cdot i + C_2 \cdot i^2) \bmod m$, where $h'(k) = k \bmod m$, k is the key, m is the table size, and $i = 0, 1, 2, \ldots$ is the probe number.

Demo with double hashing

In a double hashing scheme, $h_1(k) = k \bmod 11$ and $h_2(k) = 1 + (k \bmod 7)$ are the auxiliary hash functions. The hash table size m is 11. The hash function for the i-th probe in the open address table is $h(k,\, i) = (h_1(k) + i \cdot h_2(k)) \bmod m$. Insert the following keys into the hash table in the given order: 63, 50, 25, 79, 67, 24, 89, 44, 75, 38.

Table size: $m = 11$, $h_1(k) = k \bmod 11$, $h_2(k) = 1 + (k \bmod 7)$
Probe function: $h(k, i) = \big(h_1(k) + i \cdot h_2(k)\big) \bmod 11$
Key Sequence: 63, 50, 25, 79, 67, 24, 89, 44, 75, 38

Code for demo

```
#include <iostream>
using namespace std;

#define NIL -1

int hashInsert(int*, int, int);
int hashSearch(int*, int, int);
void displayTable(int*, int);

int main()
{
    int m, n, key,result;
    cout << "Enter hash table size: ";
    cin >> m;
    int* T =new int[m];
    for (int i =0; i < m; i++)
        T[i] = NIL;
    cout << "Enter number of keys: ";
    cin >> n;
    for (int i= 0;i < n; i++)
    {
        cout << "Key " << i+1 << ": ";
        cin >> key;
        cout << "\nInsert key "<< key << endl;
        result = hashInsert(T, m, key);
        if (result == NIL){
            cout << "Hash table overflow! Cannot insert " << key << endl;
            break;
        }
    }
    cout << "\nHash Table:" << endl;
    displayTable(T, m);
    cout << "\nEnter key to search: ";
    cin >> key;
    result = hashSearch(T, m, key);
    if (result == NIL)
```

```
        cout << "Key " << key << " not found" << endl;
    else
        cout << "Key " << key << ": found at slot " << result << endl;
    delete[] T;
    return 0;
}

int hashInsert(int* T, int m, int key)
{
    int i = 0, q;
    int h1 = key % 11;
    int h2 = 1 + (key % 7);
    do
    {
        q = (h1 + i * h2) % m;
        if (i == 0)
            cout << "h(k,0) = " << q << endl;
        else
            cout << "Attempt " << i + 1 << ": i = " << i << endl
                 << "h(k," << i << ") = (" << h1 << " + " << i << " * " << h2 << ") mod " << m << " =
                      " << q << endl;
        if (T[q] == NIL)
        {
            if (i == 0)
                cout << "Slot " << q << " is empty. Place " << key << " at index " << q << "." <<
                    endl;
            else
                cout << "Slot " << q << " is empty. Success!" << endl
                     << "Place " << key << " at index " << q << "." << endl;
            T[q] = key;
            return q;
        }
        else
            cout << "Slot " << q << " is occupied by " << T[q] << ". Collision!" << endl;
        i++;
    } while (i != m);
    return NIL;
}

int hashSearch(int* T, int m, int key)
{
    int i = 0, q;
    int h1 = key % 11;
    int h2 = 1 + (key % 7);
    do
    {
        q = (h1 + i * h2) % m;
        if (T[q] == key)
            return q;
        i = i + 1;
    } while (T[q] != NIL && i != m);
    return NIL;
}
void displayTable(int* T,int m)
{
    for (int i = 0; i < m; i++)
    {
        if (T[i]==NIL)
            cout << "Slot " << i << ": Empty" << endl;
        else
            cout << "Slot " << i << ": " << T[i] << endl;
    }
}
```

Input/Output for demo

Enter hash table size: 11
Enter number of keys: 10
Key 1: 63
Insert key 63
h(k,0) = 8
Slot 8 is empty. Place 63 at index 8.

Key 2: 50
Insert key 50
h(k,0) = 6
Slot 6 is empty. Place 50 at index 6.

Key 3: 25
Insert key 25
h(k,0) = 3
Slot 3 is empty. Place 25 at index 3.

Key 4: 79
Insert key 79
h(k,0) = 2
Slot 2 is empty. Place 79 at index 2.

Key 5: 67
Insert key 67
h(k,0) = 1
Slot 1 is empty. Place 67 at index 1.

Key 6: 24
Insert key 24
h(k,0) = 2
Slot 2 is occupied by 79. Collision!
Attempt 2: i = 1
h(k,1) = (2 + 1 * 4) mod 11 = 6
Slot 6 is occupied by 50. Collision!
Attempt 3: i = 2
h(k,2) = (2 + 2 * 4) mod 11 = 10
Slot 10 is empty. Success!
Place 24 at index 10.

Key 7: 89
Insert key 89
h(k,0) = 1
Slot 1 is occupied by 67. Collision!
Attempt 2: i = 1
h(k,1) = (1 + 1 * 6) mod 11 = 7
Slot 7 is empty. Success!
Place 89 at index 7.

Key 8: 44
Insert key 44
h(k,0) = 0
Slot 0 is empty. Place 44 at index 0.

Key 9: 75
Insert key 75
h(k,0) = 9
Slot 9 is empty. Place 75 at index 9.

Key 10: 38
Insert key 38
h(k,0) = 5
Slot 5 is empty. Place 38 at index 5.

Hash Table:
Slot 0: 44
Slot 1: 67
Slot 2: 79
Slot 3: 25
Slot 4: Empty
Slot 5: 38

Slot 6: 50
Slot 7: 89
Slot 8: 63
Slot 9: 75
Slot 10: 24

Enter key to search: 24
Key 24: found at slot 10

Exercise

Using the same key sequence $(63, 50, 25, 79, 67, 24, 89, 44, 75, 38)$ as the demo, implement open addressing with quadratic probing using the probe function:

$$h(k,\, i) = \left(h'(k) + i^2\right) \bmod m, \quad (C_1 = 0,\ C_2 = 1)$$

with two different base hash functions: [CO-4]

- **Division method (Table size $m = 11$):** $h'(k) = k \bmod m$
- **Multiplication method (Table size $m = 16, w = 32$):** $h'(k) = (A \cdot k \bmod 2^w) \gg (w - r)$, where $m = 2^r$, $\gg$ is the bitwise right-shift operator, and A is an odd integer in the range $2^{w-1} < A < 2^w$. (choose $A = 2654435769$).

1. Modify the provided demo code to implement the division method and print the output as demo code.
2. Modify the provided demo code to implement the multiplication method and print the output as a demo.
3. For both methods, compare the number of collisions corresponding to each key and comment which method you think is good.

Rubric

Criteria	Marks
Correct `hashInsert` and `hashSearch` for division method	2
Correct output for division method	1
Correct `hashInsert` and `hashSearch` for multiplication method	3
Correct output for multiplication method	1
Collision comparison table for both methods	2
Conclusion on which method is better and why	1
Total	10

Note:

- **Reference:** https://en.cppreference.com/w/c.html